

Program Structures and Algorithms

Spring 2025

Team Details:

NAME	NUID
SHREYA WANISHA	002331937
SHRIYA PRATAPWAR	002309394
DHARANA KASHYAP	002055360

GITHUB LINK: <https://github.com/PSA-Final-Project/PSA-Final-Project>

SHEET LINK:

- 1) [Tic-Tac-Toe Observation Charts](#)
- 2) [Checkers Observation Charts](#)

Project Video Demonstrations

1. **Full Project Demo:**
https://drive.google.com/file/d/1N2FmsGB22_yxEspIfwPiM9DUhbaYKSvT/view?usp=sharing
2. **TicTacToe Gameplay Demo:**
<https://drive.google.com/file/d/1WMXLxa9RBpvCWiPqA28i4Ky-aIyIm2lP/view?usp=sharing>
3. **Checkers Gameplay Demo:**
https://drive.google.com/file/d/1Ws6EOGlV8NC68guWRWm7VECyBKwRdv1k/view?usp=drive_link

We Implemented two games using the Monte Carlo Tree Search Algorithm.

- 1) TicTacToe
- 2) Checkers

TicTacToe with MCTS

Our team has completed the **backend and User Interface** implementation of Monte Carlo Tree Search (MCTS) for **TicTacToe**, along with **comprehensive unit testing** to ensure correctness and reliability across all major classes. We also **developed a benchmarking** tool that runs games with increasing MCTS iterations (**doubling from 100 to 1600**) and **records both outcome and timing data**.

Implemented Features:

1. **3x3 Board**

The game is played on a standard 3x3 grid.

Each cell can be empty, or occupied by Player X or Player O.

2. **Two Players**

- Player 0: X

- Player 1: O

Turns alternate between the two players.

3. **Initial Setup**

The board starts empty.

Player X always goes first by default (configurable, if needed).

4. **Move Rules**

Each move consists of placing a mark (X or O) in an empty cell.

Moves must be within the bounds of the board and in unoccupied cells.

5. **Valid Move Generation**

For a given state, all unoccupied cells are returned as legal moves.

Valid moves are represented by (row, column) coordinates.

6. **State Transition**

The `TicTacToeState.next()` method applies a move and returns a new state immutably.

The board and current player are updated accordingly.

7. **Terminal State Detection**

A game state is terminal if:

- One player has aligned three marks in a row (horizontally, vertically, or diagonally).
- All cells are filled with no winner (draw).

8. **Winner Detection**

- If a player aligns 3 of their marks, they are declared the winner.
- If the board is full and no player has won, the result is a draw.

9. **Randomized Move Iterator (Optional)**

Moves can be returned in random order, deterministically seeded for testing, for compatibility with MCTS or AI simulations.

10. **Difficulty Levels**

The game supports multiple difficulty levels for AI opponents:

- **Easy:** Random valid moves.

- **Medium:** Prioritizes winning moves, blocks opponent's immediate wins.
- **Hard:** Uses Minimax or MCTS to evaluate best possible moves.

11. MCTS Compatibility

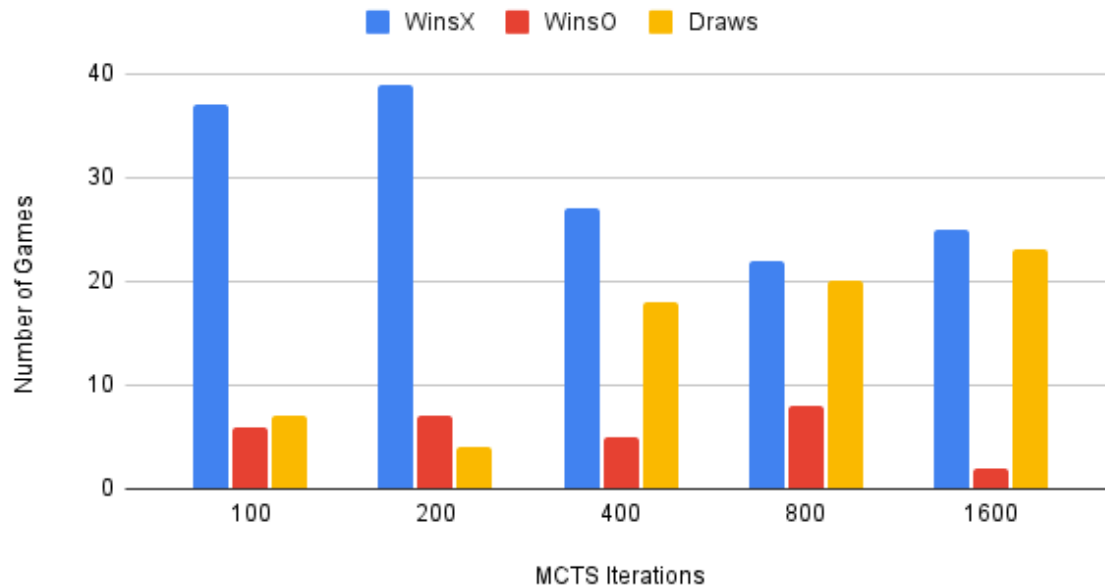
Fully integrated with the interfaces: Game, State, and Move, ensuring seamless use with MCTS agents or benchmarking tools.

Observations:

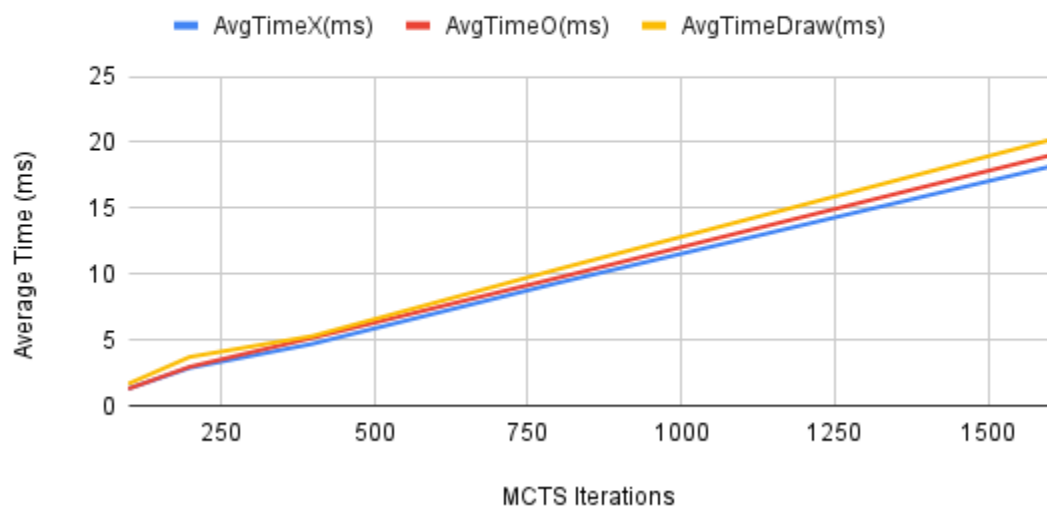
1. **Player X dominates at 100, 200, and 800 iterations** with 50 wins.
2. **All games end in draws at 400 and 1600 iterations.**
3. **Player 0 has no wins** across all iteration levels.
4. **Average time increases** with more iterations, peaking at 1600 (13 ms).

Iteration s	AvgTotalTime(ms)	WinX	AvgTimeX(ms)	Win0	AvgTime0(ms)	Draws	AvgTimeDraw (ms)
100	3	50	3.72	0	0	0	0
200	5	50	5.22	0	0	0	0
400	4	0	0	0	0	50	4.22
800	7	50	7.02	0	0	0	0
1600	13	0	0	0	0	50	13.34

Outcome Distribution vs. MCTS Iterations



Outcome-Specific Average Game Time vs. MCTS Iterations



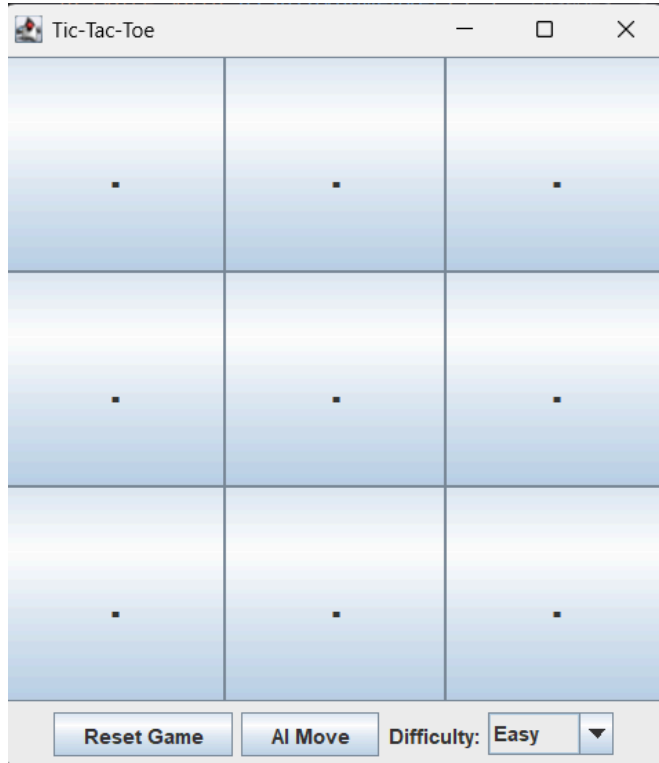
Conclusion:

Player X performs strongly at lower and mid iteration counts, while draws dominate at higher iterations, indicating improved defensive play. Player 0 consistently underperforms, and increased iterations lead to longer computation times.

GUI:

To run the GUI, run the TicTacToeGUI.java class in the repository.

Tic-Tac-Toe game user interface (UI) built with Java Swing. The interface consists of a 3x3 grid, where players place "X"(Human Player) and "O"(AI player) symbols to play the game.



Control Buttons (Bottom Panel):

- "Reset Game" Button: Restarts the game, clearing the board.
- "AI Move" Button: Triggers the AI to make a MCTS move.
- Dropdown for selecting difficulty level of game: Easy, Medium and Hard

Checkers with MCTS

Our team has completed the **backend and User Interface** implementation of Monte Carlo Tree Search (MCTS) for **Checkers**, along with **comprehensive unit testing** to ensure correctness and reliability across all major classes. We also **developed a benchmarking** tool that runs games with increasing MCTS iterations (**doubling from 100 to 1600**) and **records both outcome and timing data**.

Implemented Features:

1. 8x8 Board

The game is played on a standard 8x8 grid.

Only black squares (i.e., playable tiles) are used for moving pieces.

2. Two Players

Player 0: White

Player 1: Black

The game alternates turns between them.

3. Initial Setup

Each player starts with 12 pieces, placed on the first 3 rows (for White) and last 3 rows (for Black) on the black squares.

4. Basic Movement Rules

Each piece moves diagonally forward by one step.

White moves up the board (row--).

Black moves down the board (row++).

5. Valid Move Generation

For a given state and player, all legal single-step moves are generated and returned.

Only moves within board bounds and into empty target squares in the piece's forward direction are allowed.

6. State Transition

The `CheckersState.next()` method creates a new game state with the move applied (immutably).

The board and piece positions are updated accordingly.

7. Terminal State Detection

A game state is terminal if:

A player has no pieces left.

A player has no valid moves.

8. Winner Detection

If one player has no pieces or valid moves, the other player is the winner.

If both run out simultaneously or are blocked, it's a draw.

9. Randomized Move Iterator

Moves are randomly ordered (but deterministically for testing, using seeded Random) to comply with MCTS expectations.

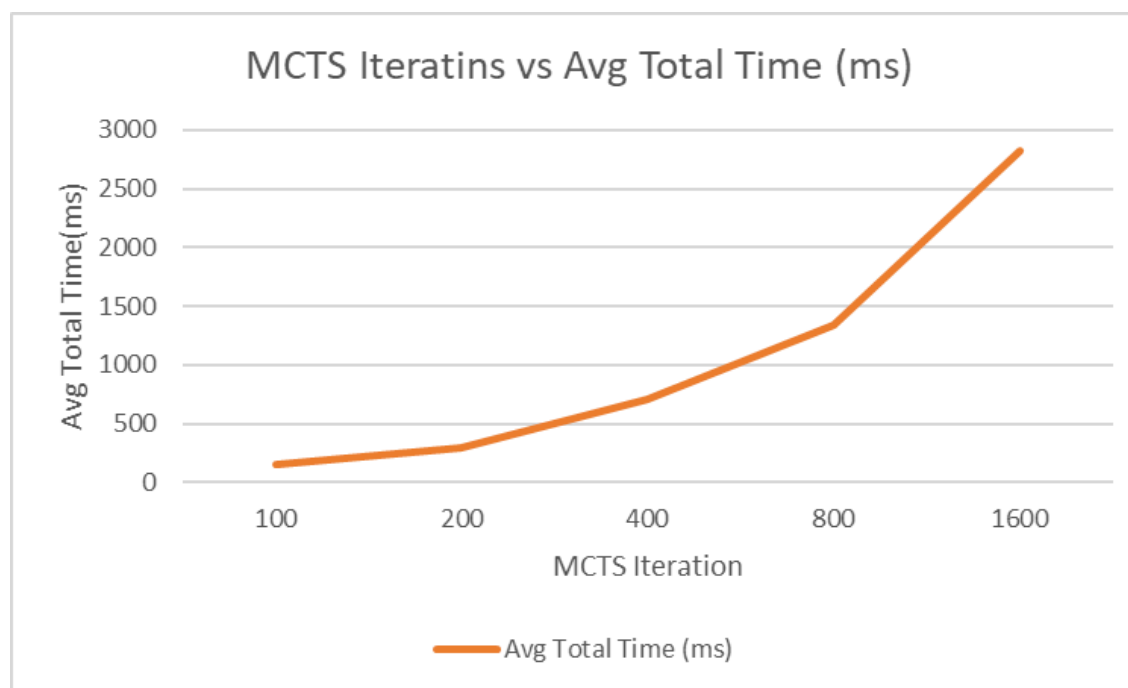
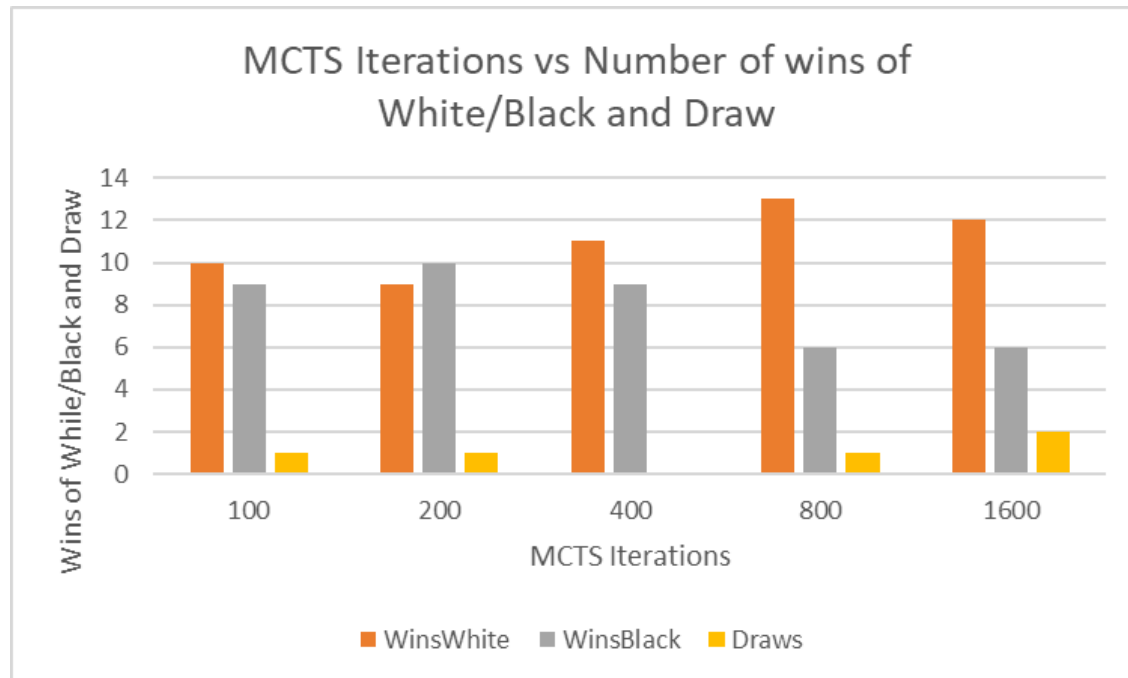
10. MCTS Compatibility

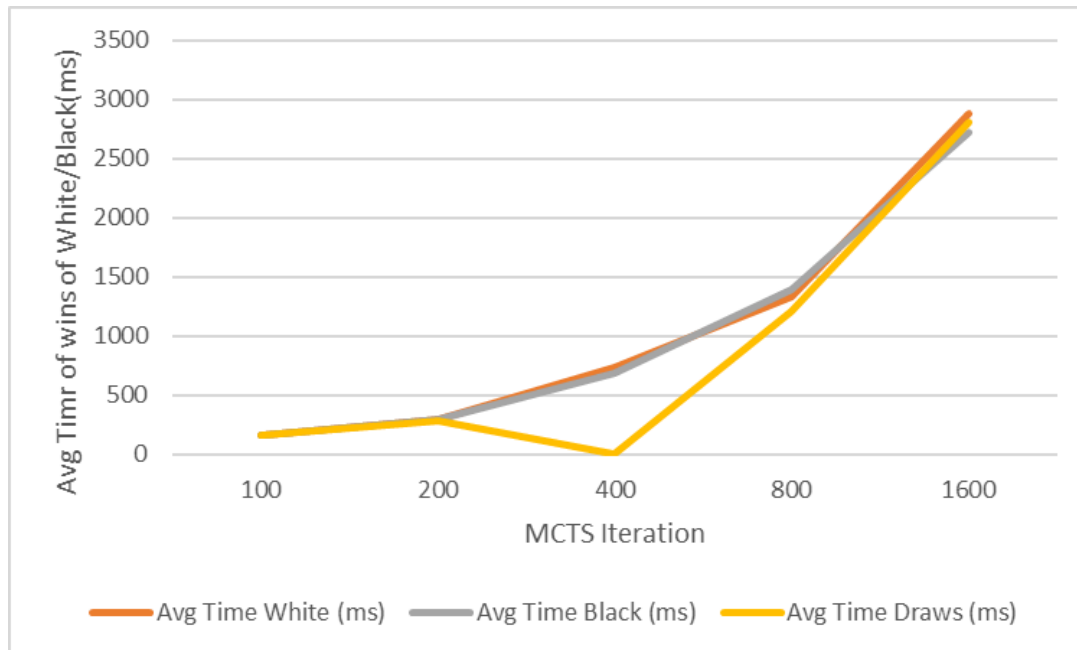
Fully integrated with the interfaces: Game, State, and Move.

Observations:

1. **Higher iterations → Higher time:** Total and per-player time increase significantly with more iterations.
2. **White wins slightly more overall**, especially at 800 and 1600 iterations.
3. **Draws are rare**, occurring only at higher iterations.
4. **Performance is fairly balanced** between players, with minor variations in average time.

Iterations	Avg Total Time (ms)	Wins White	Avg Time White (ms)	Wins Black	Avg Time Black (ms)	Draws	Avg Time Draws (ms)
100	160	10	158.3	9	162.22	1	160
200	293	9	298.22	10	290.2	1	288
400	713	11	737.09	9	684.22	0	0
800	1342	13	1326.38	6	1398.33	1	1213
1600	2828	12	2886.17	6	2718.83	2	2809.5





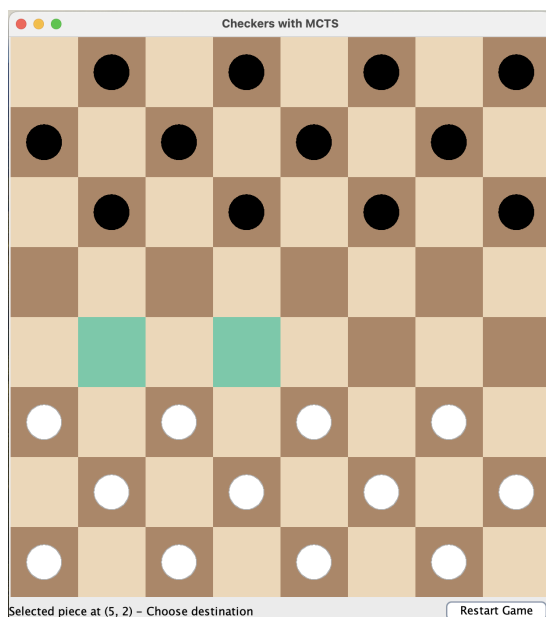
Conclusion:

MCTS performance improves with more iterations but at the cost of higher computation time. Around **800 iterations** offers a good trade-off between win rate and efficiency. Both players perform competitively, with **White slightly outperforming Black** overall.

GUI:

To run the GUI, run the CheckersGUI.java class in the repository.

Checkers game user interface (UI) built with Java Swing. The interface consists of a 8x8 grid, where players place "White"(Human Player) and "Black"(AI player) symbols to play the game.



Control Buttons (Bottom Panel):

- "Reset Game" Button: Restarts the game, clearing the board.

How to run the application :

Please follow the steps below to run both games locally:

1. Clone the repository:

Open any terminal run -

```
`git clone https://github.com/PSA-Final-Project/PSA-Final-Project`
```

2. Open the project using any Java IDE (e.g., IntelliJ, Eclipse, NetBeans).

Running TicTacToe

1. To launch the TicTacToe game UI:

Run TicTacToeGUI.java

(src/main/java/com/phasmidsoftware/dsaipg/projects/mcts/tictactoe/TicTacToeGUI.java)

2. To run the TicTacToe benchmarking tool:

Run TicTacToeBenchmark.java

(src/main/java/com/phasmidsoftware/dsaipg/projects/mcts/tictactoe/TicTacToeBenchmark.java)

Running Checkers

1. To launch the Checkers game UI:

Run CheckersGUI.java

(src/main/java/com/phasmidsoftware/dsaipg/projects/mcts/checkers/CheckersGUI.java)

2. To run the Checkers benchmarking tool:

Run CheckersBenchmark.java

(src/main/java/com/phasmidsoftware/dsaipg/projects/mcts/checkers/CheckersBenchmark.java)

NOTE : *Make sure Java 23+ is installed and set as your project SDK.*